

**DATAMITES***in association with Rubiex Internship Program***ARTIFICIAL INTELLIGENCE CAPSTONE PROJECT****PRAICP-1000 : IndiSignLang — Indian Sign Language Recognition***A Report Submitted in Partial Fulfilment of the Requirements of the **Artificial Intelligence Capstone Project***

Project Title	PRAICP-1000: IndiSignLang — Indian Sign Language Recognition
Project Code	PRAICP-1000
Team Code	PTID-AIE-JUL-26-11142
Project Type	Artificial Intelligence Capstone Project
Dataset Source	DataMites Capstone Project Dataset
Dataset Name	IndiSignLang (directories of sign images, one directory per sign)
Submitted By	Sameeksha
Organisation	DataMites / Rubiex Internship Project
Report Date	September 2026

DataMites® Innovative Solutions Pvt. Ltd.
Bengaluru, Karnataka, India

About the Author / Candidate Profile

This section presents a brief profile of the candidate who has authored and submitted this capstone project report, along with the administrative details under which the project was carried out as part of the DataMites Artificial Intelligence Engineer program delivered in association with the Rubiex Internship initiative.

Candidate Name	Sameeksha
Program	Artificial Intelligence Engineer (AIE) — DataMites
Internship Track	Rubiex Internship Project
Team Code	PTID-AIE-JUL-26-11142
Project Code	PRAICP-1000
Project Title	PRAICP-1000: IndiSignLang — Indian Sign Language Recognition
Project Type	Artificial Intelligence Capstone Project
Domain	Accessibility Technology / Computer Vision — Deep Learning (Image Classification)
Tools & Technologies	Python, TensorFlow, Keras, Keras Tuner, NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn
GitHub Repository	https://github.com/Sameekshagithub/PRAICP-1000-Indian-Sign-Language-ISL-Recognition-Using-CNN.git
Supporting Drive Link	https://drive.google.com/drive/folders/1uiFsyNDpfCWIWidHgfDLZDjr36PixxGG?usp=drive_link
Report Prepared On	September 2026

A Note on this Report

This document has been compiled as a comprehensive, stand-alone project report derived from the candidate's Jupyter Notebook submission for the PRAICP-1000 capstone assignment. It restates the problem statement, methodology, code-level design decisions, exploratory findings, model architecture, hyperparameter tuning results, quantitative results and business conclusions of the original notebook in a structured, examiner-friendly format suitable for academic evaluation and portfolio presentation. Wherever personal or organisational details (such as contact information, mentor name, or batch number) are not available in the source notebook, placeholders have been used and are clearly indicated; the candidate is encouraged to update these fields before final submission.

Declaration

I, Sameeksha, hereby declare that this project report titled "PRAICP-1000: IndiSignLang — Indian Sign Language Recognition" is a record of an original piece of work carried out by me as part of the Artificial Intelligence Capstone Project (Project Code: PRAICP-1000, Team Code: PTID-AIE-JUL-26-11142) under the DataMites Artificial Intelligence Engineer program, undertaken in association with the Rubiex Internship Project.

I further declare that the analysis, model development, results and conclusions presented in this report are based on my own work using the dataset supplied for this capstone project, and that this report has not been submitted, in whole or in part, for the award of any other certification or qualification.

All external libraries, tools (Keras Tuner) and reference material used in the course of this project have been duly acknowledged in the relevant sections of this report.

Signature: _____

Name: Sameeksha
Date: September 2026

Acknowledgement

I would like to express my sincere gratitude to DataMites® Innovative Solutions Pvt. Ltd. and the Rubiex Internship Project for providing me with the opportunity, mentorship, curriculum and dataset required to undertake this Artificial Intelligence Capstone Project on Indian Sign Language Recognition.

I am grateful to my mentors and instructors for their continuous guidance on deep learning concepts, convolutional neural network design, hyperparameter tuning methodology and evaluation techniques for multi-class image classification problems, which greatly enriched my understanding of applying Artificial Intelligence to real-world accessibility challenges.

I also wish to thank my peers and the DataMites support team for their valuable feedback during the course of this project, and my family for their constant encouragement throughout the duration of this internship and capstone submission.

Table of Contents

About the Author / Candidate Profile

Declaration

Acknowledgement

Executive Summary

1. Problem Statement & Project Objective

2. Import of Python Libraries & Tools

3. Dataset Upload & Domain Analysis

4. Basic Checks on the Dataset

5. Exploratory Data Analysis (EDA)

6. Image Preprocessing

7. Train / Validation / Test Split & Data Preparation

8. Data Augmentation

9. Model Building & Training — Baseline CNN

10. Hyperparameter Tuning

11. Final CNN Model (Tuned)

12. Model Comparison Report

13. Best Model — Detailed Evaluation

14. Predicting the Sign of a New Image

15. Saving the Model

16. Business Insights

17. Challenges Faced

18. Final Conclusion

19. Project Links

20. Ethical Considerations & Limitations

21. Future Scope & Recommendations

22. References

Appendix A: Glossary of Terms

Appendix B: Environment & Reproducibility Notes

Executive Summary

Sign language is the primary means of communication for a large population of people who are deaf or hard of hearing, yet most of the hearing population cannot read or interpret it, creating a persistent communication barrier in education, healthcare, banking and everyday public life. This capstone project, undertaken as part of the DataMites Artificial Intelligence Engineer program in association with the Rubiex Internship Project, addresses this barrier by building a Convolutional Neural Network (CNN) capable of recognising static Indian Sign Language (ISL) hand signs from photographs, as a first building block towards an assistive communication tool.

The project uses the DataMites-supplied IndiSignLang dataset, comprising 4,972 labelled images spread across 24 sign classes corresponding to the English letters A through Y (excluding J and Z, whose ISL representation requires hand motion and therefore cannot be captured in a single still photograph). The dataset exhibits a moderate class imbalance (an imbalance ratio of approximately 2.23:1 between the largest class, B, and the smallest class, H) and contains images at three distinct native resolutions, both of which were carefully addressed through the project's preprocessing, stratified-splitting and class-weighting strategy.

Two CNN models were built and rigorously compared: a baseline CNN trained with a fixed, reasonable architecture, and a final tuned CNN whose architecture and training hyperparameters (convolutional filter counts, dense-layer width, dropout rate, learning rate and batch size) were selected using Keras Tuner's RandomSearch algorithm across eight trials, evaluated purely on validation accuracy to keep the held-out test set completely unbiased. The baseline CNN achieved a strong test accuracy of 99.33% with a macro-averaged F1-score of 0.9926, while the tuned final CNN improved on this further, achieving a validation accuracy of 99.87% and a perfect test accuracy of 100.00% (macro Precision, Recall and F1-score all equal to 1.0000) on the 746-image held-out test set.

This report documents the complete workflow — from problem definition and domain analysis, through data engineering, exploratory analysis, preprocessing, augmentation, baseline modelling, systematic hyperparameter tuning, and rigorous final evaluation — and closes with business insights, a candid discussion of the project's limitations (including the caveats around a perfect test score), ethical considerations for accessibility technology, and concrete recommendations for extending this static-image classifier into a more complete, real-time, motion-aware ISL recognition system.

1. Problem Statement & Project Objective

1.1 Problem Statement

Sign language is the primary medium of communication for many people who are deaf or hard of hearing. It is a complete, structured visual language in its own right, expressed through hand shapes, hand movement, orientation and facial expression rather than speech. However, the overwhelming majority of the hearing population does not understand sign language, which creates a persistent, everyday communication barrier between deaf and hearing individuals — in classrooms, hospitals, banks, government offices, railway stations, and countless other public and private settings where quick, reliable communication matters.

India, in particular, has a large deaf and hard-of-hearing population that relies on Indian Sign Language (ISL), which is grammatically and visually distinct from American Sign Language (ASL) and other national sign languages. Despite its importance, ISL interpreters remain scarce relative to the population that needs them, and most digital assistive-communication tools available today are built around ASL rather than ISL, leaving a meaningful accessibility gap for ISL users in India.

The objective of this project is to help close part of that gap by developing a Convolutional Neural Network (CNN) based machine learning model that can look at a photograph of a hand sign from the Indian Sign Language alphabet and automatically predict which letter it represents, thereby laying the groundwork for assistive applications that can translate static ISL hand signs into text in real time.

1.2 Project Objective

The DataMites project brief for PRAICP-1000 defines two explicit goals, both of which are addressed in this project:

1. Image Preprocessing — convert raw, variable-resolution sign photographs into a clean, consistent numerical format (resized, normalised, and augmented) that a neural network can learn from effectively.
2. ML Model to Predict the Sign — build, systematically tune, and rigorously evaluate a CNN that classifies each image into its correct sign class.

To achieve these two goals, the project follows a structured, end-to-end workflow:

1. Load and explore the dataset — examining its classes, class balance, native image resolutions and representative sample images.
2. Preprocess the images (resize to a fixed size, normalise pixel values) and split them into stratified Training, Validation and Test sets.
3. Apply data augmentation (small rotations, shifts and zooms, deliberately excluding horizontal flips) to improve generalisation.
4. Build a baseline CNN, then systematically improve it using hyperparameter tuning (Keras Tuner) to obtain a final, tuned CNN.
5. Evaluate both models using Accuracy, macro-averaged Precision/Recall/F1-score, a Confusion Matrix and a full Classification Report.
6. Demonstrate end-to-end prediction on a brand-new image, and save the trained model for future reuse.

1.3 Scope of the Project

The scope of this capstone project covers the full applied machine learning lifecycle for a multi-class image classification problem: dataset acquisition and inspection, domain analysis of ISL hand signs, data cleaning, exploratory data analysis, preprocessing and augmentation, baseline model design, systematic hyperparameter search, final model training, rigorous comparative evaluation, single-image inference, model persistence, and the translation of technical results into business insights and future recommendations. The project deliberately restricts its scope to static, single-hand letter signs recognisable from one still photograph; it does not attempt to recognise words, sentences, two-handed signs, or signs that require motion (such as J and Z), which are explicitly identified as a direction for future work in Section 21.

2. Import of Python Libraries & Tools

Before any data can be loaded or modelled, the required Python libraries are imported. These fall into five broad functional categories: general-purpose data handling, dataset acquisition, image processing, deep learning and hyperparameter tuning, and performance evaluation. Table 2.1 summarises the principal libraries used in this project.

Library	Category	Purpose in this Project
os, zipfile, json, random, warnings	Core Python / OS	File-system navigation, dataset extraction, saving class-name metadata, reproducible sampling.
NumPy, Pandas	Data Handling	Numerical array operations and tabular summarisation of class counts, image dimensions and results.
Matplotlib, Seaborn	Visualisation	Plotting class distributions, sample images, training curves, and the confusion matrix heatmap.
Pillow (PIL)	Image Processing	Opening, verifying, resizing and manipulating individual sign-image files.
Requests / urllib	Dataset Acquisition	Downloading the compressed IndiSignLang dataset archive from the DataMites CDN link.
TensorFlow / Keras	Deep Learning	Building, compiling, training and saving the baseline and tuned CNN models.
Keras Preprocessing Layers	Data Pipeline	In-model data augmentation (rotation, translation, zoom) that is active only during training.
Keras Tuner (RandomSearch)	Hyperparameter Tuning	Automated, systematic search over CNN architecture and training hyperparameters.
Scikit-learn	Evaluation	Stratified train/validation/test splitting, class-weight computation, confusion matrix and classification report.

Table 2.1: Principal Python libraries used in the project and their purpose.

TensorFlow/Keras was selected as the deep learning framework because it provides a mature, high-level API for constructing custom CNN architectures with built-in preprocessing layers, alongside Keras Tuner, a companion library that automates hyperparameter search directly against Keras models. Scikit-learn complements this by supplying the standard train/test splitting utilities (with stratification support, essential given the project's class imbalance) and the classification-evaluation toolkit (confusion_matrix, classification_report) used throughout Sections 12 and 13.

3. Dataset Upload & Domain Analysis

3.1 Dataset Acquisition

The dataset used in this project is IndiSignLang (directories of sign images, one directory per sign), supplied as the official DataMites Capstone Project Dataset for the PRAICP-1000 capstone assignment, and downloaded programmatically from the archive linked below.

<https://d3ilbtixij3aepc.cloudfront.net/projects/AI-Capstone-Projects/PRAICP-1000-IndiSignLang.zip>

The dataset arrives as a ZIP archive containing one directory per sign, with each directory holding the photographs of that specific sign; the directory name therefore directly encodes the label (class) of every image inside it. The notebook's extraction routine automatically detects the folder that contains the class directories, so it works correctly even if the archive's internal layout differs slightly, and skips the download or extraction step automatically if it has already been completed in a prior run.

3.2 Domain Analysis

Each image in this dataset shows a single hand, photographed against a plain background, forming one static Indian Sign Language letter sign. The meaning of a sign is carried almost entirely by the configuration of the hand — which fingers are extended, curled, or crossed, how the thumb is positioned relative to the palm and fingers, and how the palm itself is oriented towards or away from the camera. Because these are fundamentally spatial, shape-based visual patterns, this is a natural candidate problem for a CNN, which learns edge, contour and finger-configuration features directly from raw pixels rather than relying on manually engineered geometric descriptors.

An important domain-specific constraint shapes the scope of this dataset and project: only static signs — signs that can be fully expressed by a single hand shape held still — can be recognised from one photograph. Signs that inherently require hand movement over time cannot be captured in a single still image. This is precisely why the ISL letters J and Z, which are conventionally signed with a small in-air motion (a sequential trace, similar to how they are signed in several other sign-language alphabets), are correctly absent from this dataset's 24-class label list. This is a deliberate and sensible scoping decision rooted in the physical nature of the task, not a data-quality gap.

A second important domain observation is that different ISL signs can look very similar to one another. For example, several signs are formed with a closed or near-closed fist and differ from one another only in subtle thumb placement or slight finger curl. This visual closeness between certain sign pairs is expected, a priori, to make those specific classes harder for any classifier (human or machine) to separate reliably, and this expectation is explicitly investigated later using the confusion matrix and per-class F1-scores in Section 13.

4. Basic Checks on the Dataset

Before any exploratory analysis or modelling is undertaken, a set of basic sanity checks is performed to understand the size, quality and class balance of the dataset. A structured table is first built with one row per image, recording its class label, file name, full path, width, height and colour mode; this table becomes the foundation for every subsequent basic check and EDA plot.

4.1 Dataset Size and Class List

The dataset contains a total of 4,972 images across 24 sign classes, corresponding to the English letters A through Y, with J and Z absent for the domain reason explained in Section 3.2. All images are confirmed to be RGB JPEG files.

Property	Value
Total Images	4,972
Number of Classes	24 (A–Y, excluding J and Z)
File Format	.jpg (all files)
Colour Mode	RGB (all 4,972 images)

Table 4.1: High-level dataset composition.

4.2 Corrupt / Unreadable Image Check

Every one of the 4,972 image files was opened and verified programmatically to detect files that are missing, truncated, or otherwise unreadable. This check found zero corrupt or unreadable images across the entire dataset, confirming that the dataset is clean and that every labelled file can be safely loaded for preprocessing and training without needing any exclusion or repair step.

4.3 Class-wise Image Counts and Imbalance

Counting images per class reveals that the dataset is moderately imbalanced: the smallest class (H) contains only 116 images, while the largest class (B) contains 259 images — an imbalance ratio of approximately 2.23 : 1. Table 4.2 shows the full per-class breakdown.

Sign	Count	Sign	Count	Sign	Count	Sign	Count
A	242	G	241	N	237	U	135
B	259	H	116	O	229	V	122
C	247	I	179	P	234	W	128
D	147	K	245	Q	216	X	202
E	243	L	182	R	211	Y	251
F	226	M	235	S	229		

Table 4.2: Image count per sign class (Total = 4,972; Smallest = H at 116; Largest = B at 259).

This moderate imbalance (2.23:1, considerably milder than an extreme imbalance of, say, 10:1 or more) is not severe enough to require aggressive resampling techniques such as oversampling or synthetic minority image generation, but it is significant enough to warrant deliberate countermeasures. This project addresses it in two complementary ways, detailed further in Sections 7

and 8: stratified splitting, which ensures every split (train/validation/test) preserves the same relative class proportions as the full dataset, and class weighting, which assigns proportionally higher loss penalties to under-represented classes such as H, U, V and W during training.

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to visually and statistically understand the dataset before committing to a specific CNN architecture. The EDA in this project covers the class distribution, the native resolutions of the original (pre-resize) images, one representative sample image from every one of the 24 signs, and a closer look at multiple images within a single sign class to assess within-class variation.

5.1 Class Distribution

A bar chart of image counts per sign class visually confirms the moderate imbalance already quantified in Section 4.3, with classes such as B and Y at the higher end and H, U, V and W at the lower end. This chart provides an intuitive justification for the stratified-splitting and class-weighting strategy adopted later in the project.

5.2 Original Image Resolutions

Unlike a dataset captured under one single, controlled protocol, this dataset's images were captured at three distinct native resolutions, summarised in Table 5.1.

Resolution	Orientation	Number of Images
640 × 480	Landscape	3,963
1920 × 1088	Landscape	830
1088 × 1920	Portrait	179

Table 5.1: Distribution of original (pre-resize) image resolutions and orientations.

This mixture of resolutions and orientations — including a meaningful minority of portrait-oriented images mixed in with a majority of landscape images — confirms the necessity of the uniform resizing step performed in Section 6, since a CNN's convolutional and dense layers require fixed-size, fixed-shape input tensors.

5.3 Sample Images: One per Sign

A grid displaying one representative sample image from each of the 24 sign classes was plotted for visual inspection. This confirms that the dataset consists of clear, well-lit photographs of a single hand against a plain background, and gives an initial, qualitative impression of how visually distinct — or, in some cases, visually similar — certain sign pairs are from one another.

5.4 Within-Class Variation

To assess how much natural variation exists within a single sign class, multiple sample images from one selected sign were plotted side by side. This reveals meaningful variation in hand size, skin tone, camera angle, position within the frame, and image sharpness (with some images showing mild motion blur). This natural, real-world variability is an important justification for the data augmentation strategy adopted in Section 8, since a model that has only ever seen perfectly centred, sharp images would likely struggle to generalise to the full diversity of real-world input photographs.

5.5 EDA Summary (Conclusion)

1. The dataset contains 24 sign classes (the letters A–Y without J and Z, which are not present because they require motion) and 4,972 images, all stored as RGB JPEG files, with no corrupt or unreadable files found.
2. The classes are moderately imbalanced: the smallest class (H) has 116 images while the largest (B) has 259, an imbalance ratio of roughly 2.2 : 1. This is not extreme, but it is addressed later using stratified splitting and class weights during training.
3. The images come in three different resolutions — 640×480 (3,963 images), 1920×1088 (830 images) and 1088×1920 portrait (179 images). A CNN needs a fixed input size, confirming the need for the resizing step in preprocessing.
4. The sample images show a single hand on a plain, light background, which is helpful because the background carries little confusing information. However, images within the same class vary in hand size, skin tone, camera angle, position in the frame and sharpness, supporting the use of data augmentation.
5. Some signs are visually very close to each other (for example, signs formed with a closed fist or curled fingers). These are expected to be the hardest to separate and are examined later in the confusion matrix (Section 13).

6. Image Preprocessing

Image preprocessing is the first of the two explicit goals defined in the DataMites project brief for PRAICP-1000. A CNN cannot directly accept images of differing sizes, and it learns most effectively when its numerical inputs sit on a small, consistent scale. Accordingly, two preprocessing steps are applied to every single image in the dataset, with no exceptions.

6.1 Resizing

Every image is first converted to RGB (guaranteeing exactly three colour channels for every image, regardless of its original mode) and then resized to a fixed 128×128 pixel size, producing a consistent (128, 128, 3) array (height $\times$ width $\times$ Red/Green/Blue channels) for every image in the dataset. A target size of 128×128 was deliberately chosen as a balance point: it is large enough to preserve the fine finger-level detail that distinguishes one sign from another, while remaining small enough to keep both training time and memory consumption practical on modest, non-GPU hardware.

6.2 Normalisation

Raw pixel values naturally range from 0 to 255 (standard 8-bit colour depth). Every pixel value is divided by 255 so that all inputs lie within the range 0 to 1. This rescaling is standard practice in deep learning: it keeps the numerical scale of the network's inputs small and consistent, which makes gradient-based optimisation (as used by the Adam optimizer in this project) noticeably more stable and helps the network converge faster during training.

6.3 Consistency Between Training and Inference

A single, shared `preprocess_image()` function is defined once and reused everywhere an image needs to be prepared for the model — during initial dataset loading and training, and later during live prediction on a brand-new image in Section 14. This design guarantees that a new, previously unseen photograph is processed in exactly the same way as every training image (identical resize target, identical channel handling, identical normalisation), which is essential: if training and inference preprocessing ever diverged, the model would effectively be asked to make predictions on data that looks statistically different from what it learned on, silently degrading real-world accuracy.

6.4 A Note on Aspect Ratio

Because the dataset mixes landscape images (640×480 and 1920×1088) with a minority of portrait images (1088×1920), resizing every image to a fixed square of 128×128 necessarily and slightly distorts the original aspect ratio for some images. This trade-off was accepted deliberately: since the exact same resizing transformation is applied uniformly to every training image and to every future prediction image, the CNN simply learns directly from the resized (slightly aspect-adjusted) versions, and is never evaluated against the undistorted originals it was not trained on. In effect, the model learns in a consistent "squashed" coordinate space, which is internally coherent even though it departs slightly from the true original geometry of a subset of the images.

6.5 Resulting Data Tensor

Property	Value
Final Image Shape	(128, 128, 3)
Full Dataset Tensor Shape (X)	(4972, 128, 128, 3)

Property	Value
Full Label Vector Shape (y)	(4972,)
Pixel Value Range (after normalisation)	0.0 to 1.0
Data Type	float32
Approximate Memory Footprint of X	$\approx$ 978 MB

Table 6.1: Final preprocessed data tensor dimensions and memory footprint.

7. Train / Validation / Test Split & Data Preparation

With every image resized and normalised into a single, consistent tensor, the next step is to divide the dataset into three separate, purpose-specific sets.

7.1 Split Ratios and Purpose

1. Training set (70%) — the images the CNN actually learns its filter weights and biases from.
2. Validation set (15%) — used during development to monitor training progress, trigger early stopping, and choose the best hyperparameters (Section 10), without ever touching the test set.
3. Test set (15%) — kept completely untouched until the very end of the project, used only once, to obtain a final, unbiased estimate of how the selected model performs on genuinely unseen images.

7.2 Stratified Splitting

Given the moderate class imbalance identified in Section 4.3 (an imbalance ratio of approximately 2.23:1), a plain random split could, by chance, under-represent a minority class such as H, U, V or W in one of the three sets. To prevent this, the split is stratified: every one of the three sets is constructed so that it preserves, as closely as possible, the same relative proportion of each of the 24 sign classes as the full dataset. Table 7.1 shows the resulting split sizes.

Split	Number of Images	Approximate Proportion
Training	3,480	70.0%
Validation	746	15.0%
Test	746	15.0%
Total	4,972	100.0%

Table 7.1: Final stratified train / validation / test split sizes.

The original file paths are split alongside the image tensors and labels, so that any individual test-set image can always be traced back to its original file on disk — a capability that is used later, in Section 13.4, to display the actual misclassified images (where any exist) for qualitative review.

7.3 Class Weights

In addition to stratified splitting, per-class weights are computed from the training set so that mistakes on smaller classes (such as H, U, V and W) are penalised more heavily during training than mistakes on larger classes (such as B or Y). A class-weight value above 1.0 indicates an under-represented class that receives extra importance; a value below 1.0 indicates an over-represented class. Table 7.2 shows a representative sample of the computed weights, highlighting the smallest and largest classes.

Class	Weight	Class	Weight	Class	Weight
H (smallest class)	1.79	D	1.41	A	0.86
V	1.71	L	1.14	C	0.84
W	1.61	I	1.16	K	0.84

Class	Weight	Class	Weight	Class	Weight
U	1.53	X	1.03	Y	0.82
				B (largest class)	0.80

Table 7.2: Selected class weights (full weight dictionary spans all 24 classes).

These weights are passed into the `class_weight` argument of every model's `.fit()` call, ensuring the loss function accounts for the moderate imbalance identified during EDA, in combination with the stratified splitting strategy above.

8. Data Augmentation

With only about 3,480 training images spread across 24 classes (an average of roughly 145 images per class), a CNN of any meaningful depth can quite easily memorise the specific training photographs rather than learning the general, underlying shape of each sign — a failure mode known as overfitting. Data augmentation directly counters this by showing the network a slightly modified version of each training image at every epoch, rather than the exact same pixels every single time.

8.1 Augmentation Techniques Used

1. Rotation of up to approximately ± 14 degrees, simulating a hand that is tilted rather than held perfectly upright.
2. Translation (shift) of up to 10% of the image width and height, simulating a hand that is not perfectly centred in the camera frame.
3. Zoom of up to 10%, simulating a hand held closer to, or further from, the camera.

8.2 A Deliberate Omission: No Horizontal Flipping

Horizontal flipping is a very common and often default augmentation technique in general-purpose image classification, but it is deliberately excluded from this project. The reason is domain-specific and important: a mirrored (horizontally flipped) photograph of a hand effectively turns a left hand into a right hand, or vice versa, and for many ISL signs, flipping the hand in this way could plausibly turn one valid sign into what looks like an entirely different (or invalid) hand configuration. Since flipping cannot be guaranteed to preserve the true label of the sign, including it as an augmentation would risk teaching the model incorrect, misleading examples. This is a clear illustration of why augmentation choices must always be grounded in domain understanding rather than applied as a generic, one-size-fits-all default.

8.3 Implementation as In-Model Layers

Rather than augmenting images as a separate, offline preprocessing pass, the augmentation transformations are implemented using Keras' built-in preprocessing layers and placed directly at the very start of the model architecture itself (see Section 9). These layers are automatically active only while the model is in training mode; they are automatically and transparently switched off during validation, testing, and live prediction. This design guarantees that every reported validation and test metric in this report is always computed on the original, unaltered images, keeping the evaluation completely fair and interpretable, while the model still benefits from augmented variety purely during the learning phase.

9. Model Building & Training — Baseline CNN

A baseline model is deliberately built and trained first, before any hyperparameter tuning is attempted. Its purpose is to establish a fair reference score: if the later, systematically tuned model cannot outperform this simple, reasonably-configured baseline, that would indicate the tuning process added no real value. Building a baseline first, and reporting both models' results honestly and side by side (Section 12), is good applied-machine-learning practice, since it lets the benefit of tuning be judged from evidence rather than assumption.

9.1 How the CNN Works, Layer by Layer

1. Convolution layers (Conv2D): small 3×3 filters are slid across the image. The first convolutional layer detects simple, low-level features such as edges and lines; deeper convolutional layers combine these into more complex finger shapes and the overall hand structure.
2. Max pooling (MaxPooling2D): shrinks the spatial size of the feature maps while retaining their strongest activations, which makes the model faster to train and less sensitive to the exact pixel position of a finger or edge.
3. Flatten: converts the final, multi-channel 2-D feature maps produced by the convolutional stack into a single, one-dimensional feature vector.
4. Dense layer + Dropout: the flattened features are combined by one or more fully connected (Dense) layers to make the actual classification decision; Dropout randomly deactivates a fraction of neurons during training only, which reduces overfitting by preventing neurons from co-adapting too closely to one another.
5. Softmax output layer: produces one probability value for each of the 24 sign classes, all summing to 1.0. The sign with the single highest probability is taken as the model's prediction.

9.2 Baseline Architecture and Configuration

Architecture / Training Property	Value
Input Layer	Data Augmentation block (active only during training)
Convolutional Stack	Conv2D(32) → MaxPool → Conv2D(64) → MaxPool → ... → Flatten
Classification Head	Dense layer + Dropout → Dense(24, activation='softmax')
Total Parameters	3,307,736
Trainable Parameters	3,307,736
Non-Trainable Parameters	0
Optimizer	Adam
Loss Function	Sparse Categorical Cross-Entropy
Max Epochs (with Early Stopping)	30
Class Weights Applied	Yes (Section 7.3)

Table 9.1: Baseline CNN architecture and training configuration.

The model is compiled using the Adam optimizer together with sparse categorical cross-entropy as its loss function — "sparse" because the labels used are plain integers from 0 to 23 (one per sign class) rather than one-hot encoded vectors, which is a more memory-efficient representation for a 24-class problem. Two Keras callbacks are used throughout training: EarlyStopping, which halts training once validation loss stops improving and automatically restores the best-performing weights observed, and ReduceLROnPlateau, which automatically lowers the learning rate once training progress stalls, allowing the optimizer to take smaller, more careful steps as it approaches a good solution.

9.3 Baseline Training Behaviour and Results

Training accuracy rose steadily across epochs, from approximately 15.1% after the first epoch to over 90% within eight epochs, while validation accuracy climbed even faster, reaching 95.6% by epoch 7 — evidence that the class-weighted, augmented training pipeline was learning effectively and generalising well to unseen validation images, rather than merely memorising the training set. The final baseline results, measured once training completed, are shown in Table 9.2.

Metric	Value
Baseline CNN — Validation Accuracy	0.9946
Baseline CNN — Test Accuracy	0.9933

Table 9.2: Baseline CNN validation and test accuracy.

A baseline test accuracy of 99.33% is already a strong result for a 24-class problem, and it sets a high bar for the subsequent hyperparameter-tuning stage (Section 10) to clear.

10. Hyperparameter Tuning

10.1 Parameters vs. Hyperparameters

A CNN learns its internal weights and biases automatically during training, through backpropagation and gradient descent — these are its parameters. However, several structural and training-process choices must be fixed before training even begins, such as how many filters a convolutional layer should have, how wide a dense layer should be, what dropout rate to use, what learning rate to start at, and what batch size to train with. These are hyperparameters, and unlike ordinary parameters, they are not learned by gradient descent. Rather than simply guessing reasonable-sounding hyperparameter values (as the baseline model in Section 9 effectively did), this project systematically searches for better ones, selecting the combination that performs best on the validation set — the test set is deliberately never used for this selection process, so it remains a fair, unbiased judge of the final chosen model.

10.2 Search Space

Six hyperparameters were included in the search space, summarised in Table 10.1.

Hyperparameter	Values Tried
Filters in the 1st convolution layer	32, 64
Filters in the 2nd convolution layer	64, 128
Dense layer units	128, 256
Dropout rate	0.3, 0.4, 0.5
Learning rate	0.001, 0.0001
Batch size	16, 32

Table 10.1: Hyperparameter search space (a full grid would require $2 \times 2 \times 2 \times 3 \times 2 \times 2 = 96$ combinations).

Exhaustively training and evaluating all 96 possible combinations from this grid would be prohibitively slow given the training time observed per epoch. To make the search tractable, Keras Tuner's RandomSearch algorithm is used instead: it randomly samples a fixed, manageable number of combinations (trials) from the search space, trains each candidate briefly, and ranks all trials by validation accuracy. The same data-augmentation block and the same class weights used for the baseline model are applied consistently in every trial, ensuring the eventual comparison between the tuned model and the baseline in Section 12 remains fair and like-for-like.

10.3 Tuning Results

A total of 8 trials were completed, together taking approximately 5 hours and 28 minutes of search time on the available (non-GPU) hardware. The full ranked results are shown in Table 10.2.

Rank	Batch Size	Filters-1	Filters-2	Dense Units	Dropout	Learning Rate	Val. Accuracy
1	32	32	64	256	0.3	0.0010	0.9960
2	32	32	128	128	0.4	0.0010	0.9933
3	16	32	128	256	0.5	0.0010	0.9906

Rank	Batch Size	Filters-1	Filters-2	Dense Units	Dropout	Learning Rate	Val. Accuracy
4	16	32	128	128	0.4	0.0010	0.9893
5	16	32	64	256	0.4	0.0001	0.9879
6	32	32	64	128	0.5	0.0010	0.9705
7	16	32	128	128	0.4	0.0001	0.9638
8	32	32	64	128	0.5	0.0001	0.8968

Table 10.2: All 8 tuning trials, ranked by validation accuracy (highest first).

10.4 Best Hyperparameters Selected

Trial 1 achieved the highest validation accuracy (0.9960) and its hyperparameters were therefore selected as the best hyperparameters, to be used to build the final CNN in Section 11. These values come directly from this notebook's own tuning run on this specific dataset, rather than being a pre-decided, generic answer.

Hyperparameter	Selected Value
Filters (convolution layer 1)	32
Filters (convolution layer 2)	64
Dense layer units	256
Dropout rate	0.3
Learning rate	0.001
Batch size	32

Table 10.3: Best hyperparameters selected by Keras Tuner (on validation accuracy).

11. Final CNN Model (Tuned)

The RandomSearch tuner in Section 10 only trained each of its 8 candidate configurations briefly, purely to rank them relative to one another. The final model is therefore rebuilt from scratch using the best hyperparameters identified (Table 10.3), and trained again, for longer and more thoroughly, on the full training set — still monitored throughout by the same EarlyStopping and ReduceLROnPlateau callbacks used for the baseline model, with a higher maximum epoch budget of 40 (compared to 30 for the baseline) to allow the larger, more expressive architecture to fully converge.

Architecture / Training Property	Value
Filters (convolution layer 1)	32
Filters (convolution layer 2)	64
Dense Layer Units	256
Dropout Rate	0.3
Learning Rate	0.001
Batch Size	32
Total Parameters	6,522,200
Trainable Parameters	6,522,200
Non-Trainable Parameters	0
Max Epochs (with Early Stopping)	40

Table 11.1: Final tuned CNN architecture and training configuration.

The final tuned CNN's total parameter count (6,522,200) is roughly double that of the baseline model (3,307,736), driven mainly by the wider 256-unit dense layer selected by the tuner. Training progressed rapidly and smoothly: training accuracy rose from 25.0% after epoch 1 to over 91% by epoch 8, while validation accuracy climbed even faster, reaching 97.3% by epoch 6 and 98.4% by epoch 7, with validation loss falling steadily and consistently throughout — a healthy training pattern that shows no signs of erratic overfitting during the observed epochs.

Metric	Value
Final Tuned CNN — Validation Accuracy	0.9987
Final Tuned CNN — Test Accuracy	1.0000

Table 11.2: Final tuned CNN validation and test accuracy.

The final tuned CNN improves on the baseline on both validation accuracy (99.87% versus 99.46%) and, notably, achieves a perfect 100.00% accuracy on the held-out test set. This striking result, and the important caveats that must accompany it, are discussed in full in Sections 12, 13 and 20.

12. Model Comparison Report

The baseline CNN (Section 9) and the tuned final CNN (Section 11) are compared side by side on both the validation set and the test set, using overall Accuracy together with macro-averaged Precision, Recall and F1-score. Macro averaging computes each metric independently for every one of the 24 sign classes and then averages the 24 results with equal weight, regardless of how many images each class has — which is the appropriate choice for this moderately imbalanced, 24-class dataset, since it prevents the large classes (like B or Y) from dominating the reported score at the expense of the small classes (like H, U, V or W).

Model / Split	Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)
Baseline CNN — Validation	0.9946	0.9944	0.9932	0.9937
Baseline CNN — Test	0.9933	0.9936	0.9921	0.9926
Final Tuned CNN — Validation	0.9987	0.9980	0.9978	0.9979
Final Tuned CNN — Test	1.0000	1.0000	1.0000	1.0000

Table 12.1: Baseline vs. Final Tuned CNN, compared on validation and test splits.

12.1 Selecting the Best Model

Consistent with sound machine learning practice, the best model is selected using validation accuracy, not test accuracy — the Final Tuned CNN's validation accuracy of 0.9987 comfortably exceeds the Baseline CNN's 0.9946, so the Final Tuned CNN is selected as the best model. Crucially, this selection decision is made without ever looking at either model's test-set performance, which means the test scores reported afterwards remain a genuinely honest, unbiased estimate of how the selected model performs on images it has never influenced any decision about.

On the test set, the selected Final Tuned CNN achieves a perfect accuracy of 1.0000 (100.00%), with macro Precision, Recall and F1-score all equal to 1.0000 as well — meaning it correctly classified every single one of the 746 test images, across all 24 classes, with no exceptions. This is an unusually strong outcome, and Sections 13 and 20 examine it in more depth, including a candid discussion of what a perfect test score does, and does not, tell us about a model's real-world readiness.

12.2 Did Tuning Help?

Because both models were evaluated identically, the benefit of the tuning process in Section 10 can be judged directly from the evidence in Table 12.1, rather than assumed: the Final Tuned CNN improved test accuracy from 99.33% to 100.00%, and macro F1-score from 0.9926 to 1.0000. While the baseline model was already highly accurate, the systematic hyperparameter search still delivered a measurable, evidence-backed improvement, closing essentially all of the small remaining gap left by the baseline configuration.

13. Best Model — Detailed Evaluation

The selected Final Tuned CNN is evaluated in further depth on the untouched test set, using a full per-class classification report, a confusion matrix, per-sign F1-score analysis, and a review of any misclassified images.

13.1 Classification Report

The complete classification report for the Final Tuned CNN on all 746 test images shows a precision, recall and F1-score of 1.00 for every one of the 24 sign classes, with support (test image count per class) ranging from 17 images (class H, the smallest class) to 39 images (class B, the largest class). The overall test accuracy across all 746 images is 1.00 (100%), and both the macro-averaged and weighted-averaged Precision, Recall and F1-score are all 1.00.

Class	Precision	Recall	F1-Score	Support
A	1.00	1.00	1.00	36
B	1.00	1.00	1.00	39
C	1.00	1.00	1.00	37
D	1.00	1.00	1.00	22
E	1.00	1.00	1.00	36
F	1.00	1.00	1.00	34
G	1.00	1.00	1.00	36
H	1.00	1.00	1.00	17
I	1.00	1.00	1.00	27
K	1.00	1.00	1.00	37
L	1.00	1.00	1.00	28
M	1.00	1.00	1.00	35
N – Y (remaining 12 classes)	1.00	1.00	1.00	$\Sigma = 351$
Overall Accuracy			1.00	746
Macro Average	1.00	1.00	1.00	746
Weighted Average	1.00	1.00	1.00	746

Table 13.1: Per-class classification report for the Final Tuned CNN on the test set (classes N–Y condensed for brevity; every individual class scored 1.00 across all three metrics).

13.2 Confusion Matrix

The 24×24 confusion matrix for the test set is perfectly diagonal: every one of the 746 test images was placed in its correct row-and-column cell, with a value of zero in every off-diagonal cell. In other words, no sign was confused with any other sign, including the visually similar closed-fist sign pairs flagged as a likely source of difficulty during the domain analysis in Section 3.2 and the EDA in Section 5.5.

13.3 Per-Sign F1-Scores and Most-Confused Pairs

Because every class achieved a perfect F1-score of 1.00, there is no meaningful ranking between a "weakest" and "strongest" sign in this test run — the five nominally "weakest" signs (A, B, C, D, E, each at 1.00) and the five "strongest" signs (U, V, W, X, Y, each at 1.00) are, in this particular evaluation, indistinguishable in performance. Correspondingly, there are no confused sign pairs to report for this test set.

13.4 Misclassified Images

A dedicated review step was run to collect and display any misclassified test images. It found zero misclassified images out of 746 — the confusion matrix is perfectly diagonal, so there are no error cases available for qualitative review in this particular test run.

13.5 Interpreting a Perfect Test Score

A perfect 100% test accuracy is an excellent outcome on paper, but it is important to interpret it with appropriate scientific caution rather than at face value, and this report does so deliberately and transparently in Section 20 (Ethical Considerations & Limitations). Several dataset-specific factors plausibly contribute to this result: the dataset's images are captured under fairly controlled, plain-background conditions with a single hand clearly in frame; the stratified split places different photographs of the same finite pool of hands/sessions across train, validation and test sets, meaning the test set may share more visual similarity with the training set (in terms of background, lighting, or the small number of individuals photographed) than a truly independent, real-world data source would; and the CNN, aided by data augmentation and class weighting, was evidently able to learn highly discriminative, robust features for this specific data distribution. A perfect score on this internal test set is best read as strong evidence that the model has fit this dataset extremely well — not as proof that it would perform equally perfectly on ISL photographs taken by new users, on new cameras, with new backgrounds and lighting, which is precisely the kind of external validation recommended in Section 21.

14. Predicting the Sign of a New Image

This section demonstrates the complete, end-to-end application working as a whole, from a raw input image to a final, human-readable prediction:

New hand image → Resize (128×128) → Normalise (0–1) → CNN Forward Pass → Probability for Each of the 24 Signs → Highest Probability → Predicted ISL Sign

The `predict_sign()` function reuses the exact same `preprocess_image()` function introduced in Section 6.3, which was also used to prepare every training image. This guarantees that a brand-new photograph supplied by a user is treated in precisely the same way as the images the model was trained on — same resize target, same channel handling, same normalisation — removing a common, subtle source of real-world deployment errors where training-time and inference-time preprocessing accidentally diverge.

The function returns three pieces of information: the single predicted sign (the class with the highest softmax probability), the model's confidence for that prediction (the raw softmax probability value, expressed as a percentage), and the top-3 most likely signs together with their individual probabilities, which gives visibility into cases where the model may be genuinely torn between two visually similar signs. It is worth stressing, as the original notebook itself notes, that this confidence value is the model's own internal estimate of certainty — it is not an external guarantee of correctness, and a confident but wrong prediction, while rare in this evaluation, remains theoretically possible on sufficiently unusual input images.

In the demonstration run included in the project notebook, four independently sampled test images (of signs Q, K, P and A) were each passed through `predict_sign()`. All four were correctly classified, each with a reported confidence of 100.0%, consistent with the perfect test-set performance reported in Sections 12 and 13.

15. Saving the Model

The selected Final Tuned CNN is saved to disk in the native Keras format (a single .keras file), together with a separate JSON file listing the 24 class names in the exact order used by the model's output layer. Saving the class-name list alongside the model weights is essential: a raw model file only outputs numeric class indices (0 to 23), and without the corresponding class-name mapping saved alongside it, those numbers would be meaningless to any future application that loads the model.

Artifact	Detail
Saved Model File	IndiSignLang_CNN_Model.keras (≈ 78.3 MB)
Saved Class Names File	IndiSignLang_class_names.json
Reload Verification	Reloaded model's predictions confirmed identical to the original in-memory model

Table 15.1: Saved model artefacts and verification result.

As a final integrity check, the saved model was reloaded from disk into a fresh session and used to regenerate predictions, which were then compared directly against the original, in-memory model's predictions. The two were found to be identical, confirming that the save/reload round-trip preserves the model exactly, with no loss of precision or configuration. This gives confidence that the saved .keras file, together with its class-names JSON, is ready to be shipped to and consumed by a separate application — for example, a web application or a mobile app — without needing to retrain the model.

15.1 Final Project Results Summary

Metric	Value
Number of Sign Classes	24
Images (Train / Validation / Test)	3,480 / 746 / 746
Input Size	$128 \times 128 \times 3$
Selected Model	Final Tuned CNN
Test Accuracy	100.00%
Test Precision (macro)	1.0000
Test Recall (macro)	1.0000
Test F1-Score (macro)	1.0000

Table 15.2: Final, official project results summary, as printed at the end of the notebook.

16. Business Insights

16.1 An Assistive Communication Tool

The trained CNN turns a photograph of a hand sign into a predicted letter within a fraction of a second. Integrated into a phone or web application, a model of this kind can act as a communication aid that helps people who do not know sign language understand ISL users, improving accessibility in schools, hospitals, banks and public offices — precisely the everyday communication barrier identified in the problem statement (Section 1.1).

16.2 Preprocessing Makes Real-World Use Possible

The raw photographs in this dataset came in three different resolutions and two different orientations (Section 5.2). Resizing and normalising every image — and, critically, applying exactly the same steps to brand-new images at prediction time (Section 14) — is precisely what allows a single trained model to handle photographs from different cameras and devices consistently, which is a prerequisite for any real deployment scenario where users will supply photographs from whatever phone or camera they happen to own.

16.3 Confusion Analysis Shows Where Future Effort Should Go

Although the confusion matrix, per-sign F1-scores, and misclassified-image review in Section 13 show a perfect result on this particular test set, this same analysis pipeline is the correct tool to identify weak spots the moment the model is evaluated on new, more diverse, real-world data (as recommended in Section 21). In a future iteration with external test data, these tools would show precisely which signs are confused with one another, allowing future data-collection and model-improvement effort to be focused on the specific hardest sign pairs rather than spread thinly across all 24 classes.

16.4 Hyperparameter Tuning Adds Measurable, Evidenced Value

The tuned model was compared against the untuned baseline on the same validation and test data (Section 12), with the tuned model improving test accuracy from 99.33% to 100.00% and macro F1-score from 0.9926 to 1.0000. Because this comparison is reported openly and transparently, the benefit of the roughly 5.5 hours invested in hyperparameter search can be judged from evidence rather than assumption — a useful template for justifying similar tuning investment in future projects.

16.5 Scope and Deployment Considerations

The model recognises static letter signs from a single image; it does not recognise ISL words, full sentences, two-handed signs, or motion-based signs (including the letters J and Z). It is best understood as a first, foundational building block. A production-grade accessibility system would need to be extended with video or sequence-based models capable of capturing hand motion over time, and would need to be validated directly with ISL users and accessibility experts before any real-world deployment, given the perfect-score caveats discussed in Section 13.5 and Section 20.

17. Challenges Faced

17.1 Images of Different Sizes and Orientations

The dataset contains 640×480 landscape photographs, 1920×1088 landscape photographs, and 1088×1920 portrait photographs, all mixed together. All were converted to a common 128×128 RGB format (Section 6). A fast JPEG "draft" decode mode was used while loading the larger images, so that the considerably larger 1920×1088 and 1088×1920 files did not disproportionately slow down the overall preprocessing pipeline.

17.2 Moderate Class Imbalance

Some signs (such as B and Y) have more than twice as many images as others (such as H, U, V and W), an imbalance ratio of roughly 2.2:1. This was handled with stratified train/validation/test splitting (Section 7.2) and class weights during training (Section 7.3), and both models are judged using macro-averaged metrics (Section 12) so that small classes count equally alongside large ones in the headline comparison.

17.3 Visually Similar Signs

Certain ISL signs differ from one another only by small changes in finger or thumb position (Section 3.2), which a small CNN could plausibly confuse. Rather than relying only on overall accuracy, the project examined the confusion matrix and classification report sign-by-sign (Section 13), which in this evaluation run showed no confusion between any sign pairs — though, as discussed in Section 13.5, this should be re-verified on more diverse external data before drawing a firm conclusion that these visually similar signs are truly "solved."

17.4 Risk of Overfitting on a Relatively Small Dataset

With around 3,480 training images spread across 24 classes, the network could plausibly memorise the training photographs rather than learning generalisable sign shapes. Data augmentation (rotation, shift, zoom), dropout, early stopping and learning-rate reduction were all used together to reduce this risk (Sections 8 and 9). Horizontal flipping was deliberately avoided, because a mirrored hand may not represent a valid example of the same sign (Section 8.2).

17.5 Avoiding Information Leakage in Model Selection

To keep the final reported score honest, both the hyperparameter search (Section 10) and the choice between the baseline and tuned model (Section 12.1) were made using only the validation set. The test set was used only once, at the very end, purely for the final evaluation reported in Section 13 — it played no role whatsoever in any decision made earlier in the project.

17.6 Memory and Compute Limits

Loading nearly 5,000 images into memory as float32 arrays, and then searching across several hyperparameter combinations, is computationally heavy, particularly on the non-GPU hardware used for this project (Appendix B). Images were stored at a modest 128×128 resolution, full-size intermediate arrays were deleted from memory immediately after the train/validation/test split was created, and the number of tuner trials, tuner epochs and (optionally) the maximum images sampled per class were all made configurable parameters in the notebook, so that the same pipeline can be run on more modest hardware by reducing these values if needed.

18. Final Conclusion

18.1 Project Objective

The objective of this project was to build a machine learning model that recognises Indian Sign Language signs from images, covering both explicit goals of the DataMites brief: image preprocessing, and an ML model to predict the sign.

18.2 Exploratory Data Analysis

EDA showed a 24-class dataset of single-hand photographs on plain backgrounds, a moderate class imbalance (about 2.2:1), three different original image resolutions, and natural visual variation between images of the same sign — findings that directly justified the project's resizing, normalisation, stratified splitting, class-weighting and data-augmentation strategy.

18.3 Modelling

A baseline CNN was built first, then Keras Tuner's RandomSearch algorithm was used to systematically select the number of convolutional filters, dense-layer units, dropout rate, learning rate and batch size, using only validation accuracy as the selection criterion. A final CNN was then trained from scratch using these best hyperparameters, and both models were compared fairly using identical metrics on identical data splits.

18.4 Outcome

The selected Final Tuned CNN's accuracy, macro precision/recall/F1-score, full classification report and confusion matrix on the unseen test set are reported in Sections 12 and 13, and summarised in the final results table in Section 15.1. The model achieved a perfect 100.00% test accuracy with macro Precision, Recall and F1-score all equal to 1.0000, correctly classifying all 746 test images across all 24 sign classes with zero confusions and zero misclassified images. The model can predict the sign of a brand-new image together with its confidence (Section 14), and has been saved (Section 15) ready for reuse in a downstream application. While this is an outstanding result on the project's own held-out test set, Sections 13.5 and 20 candidly discuss why this perfect score should be interpreted as strong evidence of an excellent fit to this specific dataset rather than a guarantee of equally perfect performance on new, independently collected ISL photographs — future work, outlined in Section 21, includes gathering more varied training photographs, exploring transfer learning with pretrained networks, extending to two-handed and motion-based signs, and building towards real-time webcam-based recognition.

19. Project Links

The following links provide access to the complete project resources, including the source code, dataset, and supporting documentation.

19.1 GitHub Repository

Access to the complete source code, Jupyter Notebook, README, and project files:

<https://github.com/Sameekshagithub/PRAICP-1000-Indian-Sign-Language-ISL-Recognition-Using-CNN.git>

19.2 Google Drive

Access to the accompanying presentation (PPT) and this report:

https://drive.google.com/drive/folders/1uiFsyNDpfCWIWidHgfdLZDjr36PixxGG?usp=drive_link

20. Ethical Considerations & Limitations

20.1 Ethical Considerations

1. **Community Involvement:** Any real assistive-communication tool for the deaf and hard-of-hearing community should be designed, tested and validated in direct collaboration with ISL users and Deaf-community organisations, rather than purely as a technical exercise disconnected from the community it aims to serve.
2. **Risk of Miscommunication:** An incorrect prediction in a sensitive setting (such as a hospital or legal context) could cause real harm through miscommunication. A production system should clearly communicate its confidence, allow easy correction, and never be positioned as a substitute for a qualified human ISL interpreter in high-stakes situations.
3. **Dataset Representativeness:** The dataset used here was captured under specific, comparatively controlled conditions (plain backgrounds, a limited pool of hands/sessions). It may not fully represent the full diversity of hand shapes, skin tones, ages, and signing styles found across India's actual ISL-using population, and should be expanded and re-validated on more diverse data before any real-world deployment.
4. **Data Privacy:** Photographs of a person's hand, especially if ever extended to include the signer's face (for facial-grammar components of ISL), are personal data. Any real deployment must handle image data responsibly, with clear consent and appropriate data-protection safeguards, consistent with applicable regulations such as India's Digital Personal Data Protection Act.

20.2 Limitations of this Project

1. **Perfect Test Score Caveat:** As discussed in Section 13.5, the Final Tuned CNN's 100% test accuracy should be interpreted with care. It most likely reflects the model's excellent fit to this specific, comparatively controlled dataset, rather than a guarantee of equally perfect performance on new photographs taken by different users, cameras, or in different lighting and background conditions.
2. **Static Signs Only:** The project recognises only static, single-hand letter signs. It explicitly does not handle the ISL letters J and Z (which require motion), nor does it handle numbers, words, full sentences, two-handed signs, or the facial expressions and non-manual markers that carry real grammatical meaning in full ISL communication.
3. **Single-Dataset Evaluation:** All models were trained and evaluated on a single dataset supplied by DataMites. Performance on an entirely independent, externally collected set of ISL photographs (from different signers, cameras, and environments) has not been tested and should not be assumed to match the results reported here.
4. **No Field Validation with ISL Users:** This project is a technical, academic proof-of-concept. It has not been evaluated with actual ISL signers or Deaf-community members, and should not be treated as a validated, ready-to-deploy accessibility product.

21. Future Scope & Recommendations

1. **External Validation Dataset:** Collect and evaluate the model against an entirely new set of ISL photographs — different signers, cameras, backgrounds and lighting — to obtain a realistic estimate of real-world accuracy beyond the current, likely-optimistic, in-distribution test score.
2. **Transfer Learning:** Explore building on top of ImageNet-pretrained backbones (such as MobileNetV2 or EfficientNet) rather than training a CNN entirely from scratch, which may further improve robustness and generalisation, particularly if the training dataset is expanded but remains moderate in size.
3. **Motion-Based Signs (J, Z and Beyond):** Extend the system with a video or sequence model (for example, a CNN combined with an LSTM, or a 3D-CNN) capable of capturing hand motion over time, to support the currently unsupported motion-based letters and, eventually, dynamic words and phrases.
4. **Real-Time Webcam Recognition:** Package the saved .keras model (Section 15) into a real-time application that captures live webcam frames, runs them through the same `preprocess_image()` and `predict_sign()` pipeline used in Section 14, and displays the predicted sign continuously, moving the project from a batch/photograph tool towards an interactive communication aid.
5. **Two-Handed and Word-Level Signs:** Extend the dataset and model beyond single-letter fingerspelling to full ISL words and common phrases, many of which involve two hands and are far more directly useful for everyday communication than letter-by-letter spelling.
6. **Explainability:** Incorporate Grad-CAM or similar visual-explanation techniques to show which regions of a hand image most influenced a given prediction, which would help build trust with users and make it easier to diagnose the model's mistakes once it is tested on more challenging, external data.
7. **Co-Design With the Deaf Community:** Involve ISL signers, interpreters and Deaf-community organisations directly in future data collection, evaluation design and user-interface design, consistent with the ethical considerations raised in Section 20.1.

22. References

1. O'Shea, K., & Nash, R. (2015). An Introduction to Convolutional Neural Networks. arXiv preprint.
2. Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
3. O'Malley, T., Bursztein, E., Long, J., et al. (2019). KerasTuner. Retrieved from the official Keras Tuner documentation.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
5. TensorFlow Developers. TensorFlow / Keras API Documentation. Retrieved from the official TensorFlow documentation site.
6. Indian Sign Language Research and Training Centre (ISLRTC). Overview of Indian Sign Language. Retrieved from the ISLRTC official website.
7. DataMites® Innovative Solutions Pvt. Ltd. Artificial Intelligence Capstone Project Guidelines, PRAICP-1000.

Appendix A: Glossary of Terms

CNN (Convolutional Neural Network): A class of deep neural networks that use convolutional layers to automatically learn spatial hierarchies of features from image data.

ISL (Indian Sign Language): The primary sign language used by the deaf and hard-of-hearing community in India, distinct in grammar and vocabulary from American Sign Language (ASL) and other national sign languages.

Static Sign: A sign that can be fully conveyed by a single, still hand shape, as opposed to a sign that requires movement over time.

Fingerspelling: Spelling out words letter-by-letter using individual hand signs, as opposed to using a single sign for an entire word.

Stratified Split: A way of dividing a dataset into subsets (e.g., train/validation/test) that preserves the same relative proportion of each class in every subset.

Class Weighting: A technique that assigns higher loss penalties to under-represented classes during training, to counter the effects of class imbalance.

Data Augmentation: The practice of applying random, label-preserving transformations (rotation, shifting, zooming, etc.) to training images to artificially increase data diversity and reduce overfitting.

Hyperparameter: A configuration value (e.g., learning rate, number of filters, dropout rate) that is set before training begins, rather than learned automatically from data.

Keras Tuner / RandomSearch: A library and algorithm that automates the search for good hyperparameter combinations by randomly sampling and evaluating candidates from a defined search space.

Softmax: An activation function used in the output layer of a multi-class classifier that converts raw model outputs into a probability distribution over all classes, summing to 1.0.

Macro Average: A way of averaging a metric (e.g., F1-score) across multiple classes by giving each class equal weight, regardless of how many samples it has.

Confusion Matrix: A table showing, for each true class, how many samples were predicted as each possible class; the diagonal shows correct predictions, and off-diagonal cells show specific confusions between classes.

Early Stopping: A regularisation technique that halts training once a monitored metric (e.g., validation loss) stops improving, to prevent overfitting.

Dropout: A regularisation technique that randomly deactivates a fraction of neurons during training to reduce overfitting and co-dependency between neurons.

Appendix B: Environment & Reproducibility Notes

Component	Detail
Programming Language	Python 3
Deep Learning Framework	TensorFlow 2.20.0 (Keras high-level API)
Hyperparameter Tuning Library	Keras Tuner 1.4.8 (RandomSearch)
Hardware Used	CPU only (no GPU acceleration available in the training environment)
Notebook Environment	Jupyter Notebook
Dataset Size (uncompressed)	≈ 84.6 MB archive; ≈ 978 MB as preprocessed float32 tensors
Hyperparameter Search Time	≈ 5 hours 28 minutes across 8 trials
Model Persistence Format	Native Keras (.keras) plus a class-names JSON file

The absence of GPU acceleration materially affected training time, with individual epochs taking on the order of 130–180 seconds each for both the baseline and final CNN. Readers reproducing this project on GPU-enabled hardware should expect substantially shorter training and hyperparameter-search times, though final model metrics should remain broadly consistent given the same data, architecture and augmentation policy, subject to normal run-to-run variance inherent in stochastic neural network training and in the random sampling used by RandomSearch.

For full reproducibility, refer to the original Jupyter Notebook (PRAICP_1000_IndiSignLang.ipynb) linked in Section 19, which contains the complete, executable source code corresponding to every section of this report, including the configurable TUNER_TRIALS, TUNER_EPOCHS and MAX_IMAGES_PER_CLASS parameters referenced in Section 17.6.